CRM Platform - Deployment Guide

Version: 1.0
Status: Draft
Last Updated: June 2026

Table of Contents

1. Purpose
2. Deployment Objectives
3. Deployment Architecture
4. Environment Strategy
5. Infrastructure Components
6. Containerization
7. Kubernetes (k3s)
8. CI/CD Pipeline
9. Infrastructure as Code
10. Configuration Management
11. Secrets Management
12. Networking
13. Storage
14. Monitoring & Observability
15. Backup & Recovery
16. Deployment Strategy
17. Rollback Strategy
18. Health Checks
19. Scaling
20. Future Enhancements

---

1. Purpose

This document defines how the CRM Platform is built, deployed, configured, monitored, and maintained across different environments.

The deployment architecture is designed to be:

- Cloud-native
- Containerized
- Highly maintainable
- Automated
- Scalable

- Cost-effective

The initial deployment targets the **AWS Free Tier** while maintaining a production-oriented architecture.

---

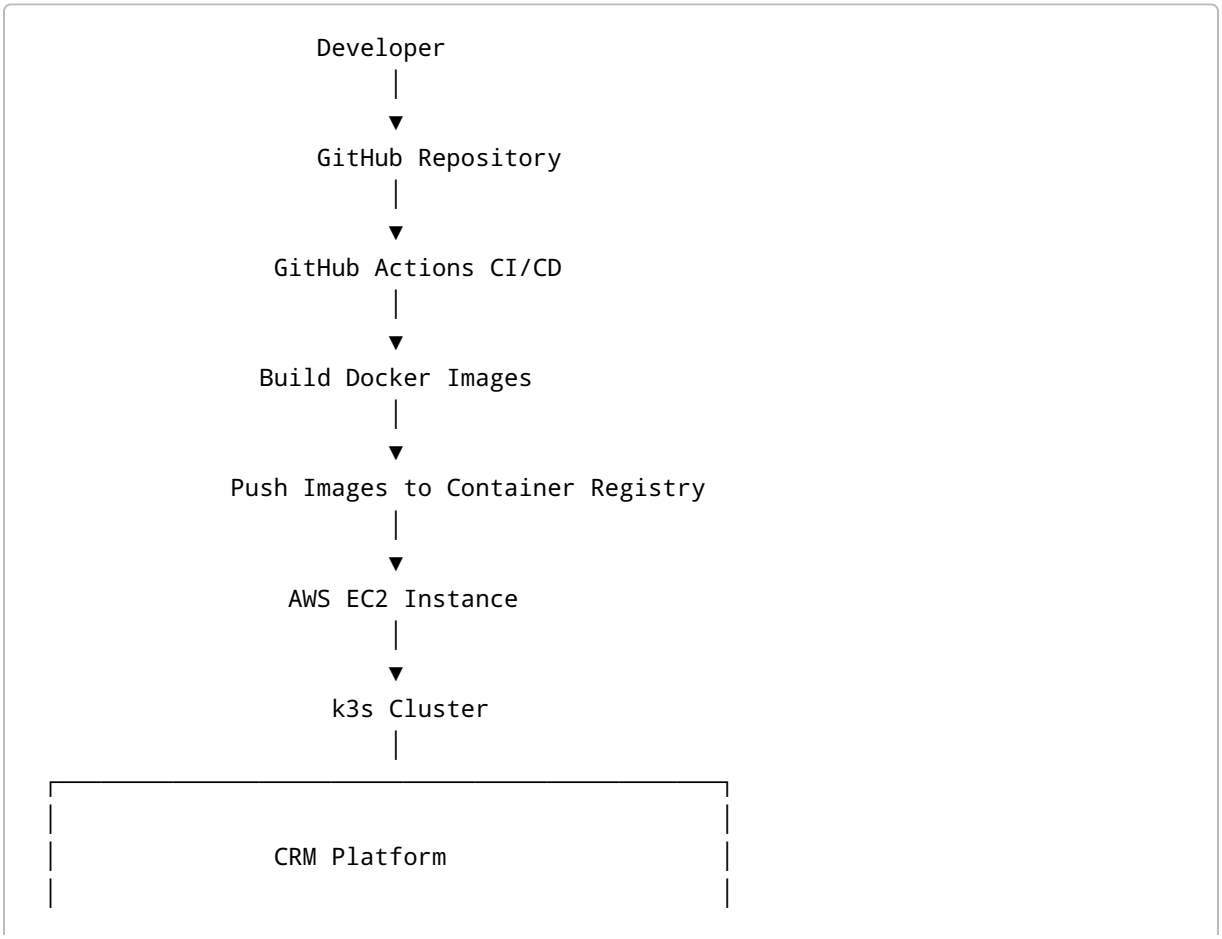
2. Deployment Objectives

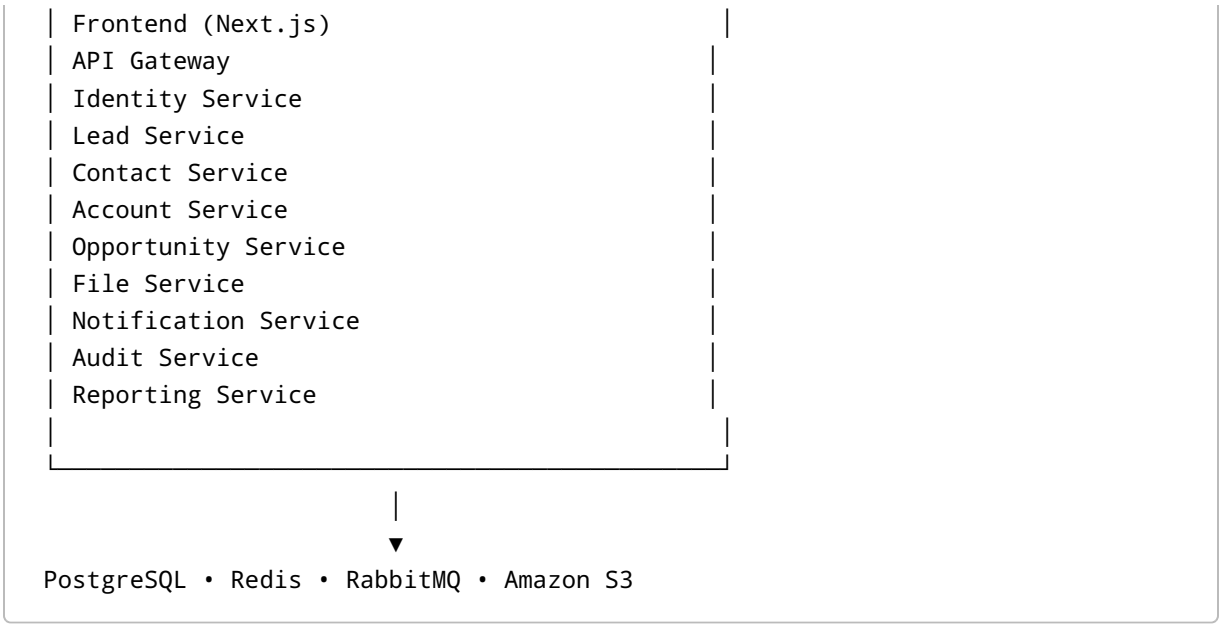
The deployment process aims to:

- Automate infrastructure provisioning
- Automate application deployment
- Minimize downtime
- Support easy rollback
- Enable horizontal scaling
- Ensure consistent environments
- Reduce manual intervention

---

3. Deployment Architecture





---

4. Environment Strategy

The platform supports multiple deployment environments.

Environment	Purpose
Local	Development
Development	Integration testing
Staging *(Future)*	Pre-production testing
Production *(Future)*	Live deployment

Initially, only **Local** and **Development** environments will be used.

---

5. Infrastructure Components

Component	Technology
Compute	AWS EC2
Orchestration	k3s
Containers	Docker
Database	PostgreSQL

Component	Technology
Cache	Redis
Message Broker	RabbitMQ
Object Storage	Amazon S3
Monitoring	Prometheus
Dashboards	Grafana
Logging	Loki
Tracing	Jaeger

---

6. Containerization

Every microservice is packaged as an independent Docker image.

Each service contains:

- Application code
- Runtime
- Dependencies
- Configuration support

Each service should have:

```
Dockerfile
.dockerignore
```

General Docker guidelines:

- Use lightweight base images
- Run containers as non-root users
- Keep images small
- Separate build and runtime stages using multi-stage builds

---

7. Kubernetes (k3s)

The platform will run on a **k3s** Kubernetes cluster hosted on AWS EC2.

Each microservice will have:

- Deployment
- Service
- ConfigMap
- Secret
- Ingress (where required)

Example deployment flow:

```
Deployment
      │
      ▼
ReplicaSet
      │
      ▼
Pods
```

Benefits:

- Self-healing
- Service discovery
- Rolling updates
- Horizontal scaling
- Simplified management

---

8. CI/CD Pipeline

The deployment pipeline is managed using **GitHub Actions**.

Pipeline stages:

```
Developer Push
      │
      ▼
GitHub Actions
      │
      ▼
Install Dependencies
      │
      ▼
Run Tests
      │
```

```
      ▼
Build Docker Image
      │
      ▼
Push Image
      │
      ▼
Deploy to k3s
```

Pipeline responsibilities:

- Lint code
- Execute unit tests
- Build containers
- Push images
- Deploy services
- Verify deployment

---

9. Infrastructure as Code

Infrastructure is provisioned using **Terraform**.

Terraform manages:

- EC2 Instance
- Networking
- Security Groups
- IAM Resources *(Future)*
- S3 Buckets
- Supporting AWS resources

Benefits:

- Version-controlled infrastructure
- Repeatable deployments
- Easier disaster recovery
- Reduced configuration drift

---

10. Configuration Management

Application configuration is externalized.

Development:

```
.env
```

Production:

- ConfigMaps
- Environment Variables

Configuration examples:

- Database URL
- Redis URL
- RabbitMQ URL
- SMTP Configuration
- JWT Settings

Configuration should never be hardcoded.

---

11. Secrets Management

Sensitive values include:

- Database passwords
- JWT signing keys
- RabbitMQ credentials
- AWS credentials
- SMTP passwords

Development:

```
.env (excluded from Git)
```

Production:

- Kubernetes Secrets

Future:

- AWS Secrets Manager

Secrets must never be committed to the repository.

---

12. Networking

Traffic flow:

```
Internet
    │
    ▼
API Gateway
    │
    ▼
Microservices
    │
    ▼
Infrastructure
```

Internal communication occurs over the Kubernetes network.

External access is limited to:

- Frontend
- API Gateway

Databases and internal services should not be publicly exposed.

---

13. Storage

Persistent storage includes:

Component	Storage
PostgreSQL	Persistent Volume
RabbitMQ	Persistent Volume
Redis *(Optional)*	Persistent Volume
Files	Amazon S3

Application containers should remain stateless.

---

14. Monitoring & Observability

Monitoring stack:

```
Prometheus
     │
     ▼
Grafana Dashboards

Loki
     │
     ▼
Centralized Logs

Jaeger
     │
     ▼
Distributed Tracing
```

Metrics collected include:

- CPU Usage
- Memory Usage
- Request Rate
- Error Rate
- Response Time
- Queue Depth

---

15. Backup & Recovery

Backup strategy:

PostgreSQL

- Daily backups
- Retention policy
- Restore testing

RabbitMQ

- Persistent queues
- Configuration backups

S3

- Versioning *(Future)*
- Lifecycle policies *(Future)*

Recovery procedures should be documented and tested periodically.

---

16. Deployment Strategy

Initial deployments use **Rolling Updates**.

Benefits:

- Minimal downtime
- Automatic replacement of old pods
- Controlled rollout

Future strategies:

- Blue/Green Deployment
- Canary Deployment

---

17. Rollback Strategy

If deployment fails:

1. Stop rollout
2. Roll back to previous image
3. Verify service health
4. Investigate root cause
5. Redeploy after resolution

Container images should be versioned and immutable.

---

18. Health Checks

Every service exposes:

```
/health
```

```
/ready

/live
```

Types:

- Liveness Probe
- Readiness Probe

These endpoints enable Kubernetes to restart unhealthy containers automatically.

---

19. Scaling

The architecture supports independent scaling of services.

Example:

```
Identity Service      x2

Lead Service          x3

Notification Service  x5

Reporting Service     x1
```

Scaling decisions are based on:

- CPU usage
- Memory usage
- Queue length
- Request volume

Future enhancements include Horizontal Pod Autoscaling (HPA).

---

20. Future Enhancements

Planned improvements include:

- Multi-node k3s cluster
- AWS Load Balancer
- Horizontal Pod Autoscaler
- Blue/Green deployments

- Canary deployments
- GitOps with Argo CD or Flux
- Service Mesh (Istio/Linkerd)
- Multi-region deployment
- Automated disaster recovery
- Cost optimization dashboards

Related Documents

- project-overview.md
- system-architecture.md
- api-standards.md
- database-design.md
- security.md